

CS3103 Project A Report

Group Information

Member	Name	SID	Email	Contribution(%)
#1 (contact person)	Lam Lok Yui	57908963	lokyuilam2-c@my.cityu.edu.hk	35
#2	NG KA HUNG	58533046	kahungng9-c@my.cityu.edu.hk	35
#3	QI Xinyuan	40162321	xinyuanqi3-c@my.cityu.edu.hk	30

Note: Please specify each team member's contribution if not all members make significant contributions to this project.

Submission due: Sunday, April 27, 2025, 11:59 pm HKT. (10 marks deducted if late for <=12 hours, 20 marks deducted if late for 12-24 hours, and so on).

Problem 1

Question 1:

1) (5 marks) When executing the three lines of code, does the variable *global_param* have the same value as the input parameter? Why? How about the variables *local_param*, and *shared_param_c*? **Please explain your answers.**

global_param: No. The parent and child processes have separate address spaces after `fork()`. After the parent process changes the value of *global_param*, the child process's *global_param* value remains 0.

local_param: No. *local_param* is a local variable of the parent process, and the child process cannot access the stack space of the parent process. *local_param* in the child process is a separate copy and retains its initial value of 0.

shared_param_c: Yes. *shared_param_c* is a pointer to the shared memory through which the parent and child processes communicate. After the parent process writes an input parameter to shared memory, the child process can read the value.

Question 2:

1) (8 marks) Have you successfully implemented the required program? If so, please attach the screenshot of the program execution results. If not, please provide potential reasons for the issues.

Please declare the chosen student ID number when answering this question.

Yes, we have successfully implemented the required program.

```
stu@CS3103:~/Desktop/project-A-2025$ ./problem1 57908963
Successfully created new semaphore!
Parent Process: Parent PID is 11518
Parent Process: Got the variable access semaphore.
Parent Process: Released the variable access semaphore.
Child Process: Child PID is 11519
Child Process: Got the variable access semaphore.
Child Process: Read the global variable with value of 57908963.
Child Process: Read the local variable with value of 57908963.
Child Process: Read the shared variable with value of 57908963.
Child Process: Released the variable access semaphore.
```

2) (7 marks) How do you handle the thread deadlocks when using the semaphore?

We initial the semaphores at the beginning of the code. Then, those semaphores are used in the addition function to prevent the thread deadlocks for mutual exclusion. Each semaphores are locked during the execution of each thread. Then, we release the semaphores for other threads to access the variables nearly immediately as the time interval of each thread is very short for concurrency.

3) (30 marks) Please attach the screenshot of the program implementation and execution. You should submit all the source files as required in the project instruction documentation and explain the relevant compile and running steps; you should make sure your program runs on virtual machine and Tutors will test your submitted code on virtual machine and verify the functionality.

```
• stu@CS3103:~/Desktop/project-A-2025$ ./problem1 57908963
Successfully created new semaphore!
Parent Process: Parent PID is 3113
Parent Process: Got the variable access semaphore.
Parent Process: Released the variable access semaphore.
Child Process: Child PID is 3114
Child Process: Got the variable access semaphore.
Child Process: Read the global variable with value of 57908963.
Child Process: Read the local variable with value of 57908963.
Child Process: Read the shared variable with value of 57908963.
Child Process: Released the variable access semaphore.
• stu@CS3103:~/Desktop/project-A-2025$
```

Open



*p1_result.txt

~/Desktop/project-A-2025

Save



1 79120185\$

Plain Text



Tab Width: 8



Ln 1, Col 10



INS

```

264 void multi_threads_run(long int input_param)
265 {
266     // Add your code here
267     pthread_t pthd[8];
268     pthread_t thread[8];
269     long int arr[8];
270
271     for(int i = 0; i < 8; i++){
272         if (sem_init(&sem_13_LamLokYui[i], 0, 1) == -1){
273             perror("initial failed");
274             exit(1);
275         }
276     }
277
278     for (int i = 0; i < 8; i++){
279         arr[i] = input_param % 10;
280         input_param /= 10;
281     }
282
283     int tracker = 0, index = 0;
284     ReverseArray(arr, 8);
285     while (tracker < 8){
286         pthd[tracker].d1 = &arr[index % 8];
287         index += 1;
288         pthd[tracker].d2 = &arr[index % 8];
289         pthread_create(&thread[tracker], NULL, addition, &pthd[tracker]);
290         sleep(1);
291         tracker++;
292     }

```

```

294     char str[8];
295     array_to_int(arr, str, 8);
296     int res = atoi(str);
297
298     saveResult("p1_result.txt", res);
299
300
301     for (int i = 0; i < 8; i++){
302         sem_destroy(&sem_13_LamLokYui[i]);
303     }
304     exit(1);
305     for (int i = 0; i < 8; i++){
306         pthread_join(thread[i], NULL);
307     }
308
309     printf("Finished the addition operation successfully!\n");
310 }

```

Explanation:

Step1: Unzip and Get the code

Step2: open VS code for compiling the code

Step3: Compile problem1.c with the provided Makefile:

“\$ make”

```
• stu@CS3103:~/Desktop/project-A-2025$ make
gcc -g -lm -c -o problem1.o problem1.c
gcc -o problem1 helpers.o problem1.o -pthread
• stu@CS3103:~/Desktop/project-A-2025$
```

-> If everything goes well, run the executable file with the command:

Examples:

“\$./problem1 57908963”

-> It takes a few seconds to compile the code.

```
• stu@CS3103:~/Desktop/project-A-2025$ ./problem1 57908963
Successfully created new semaphore!
Parent Process: Parent PID is 3342
Parent Process: Got the variable access semaphore.
Parent Process: Released the variable access semaphore.
Child Process: Child PID is 3343
Child Process: Got the variable access semaphore.
Child Process: Read the global variable with value of 57908963.
Child Process: Read the local variable with value of 57908963.
Child Process: Read the shared variable with value of 57908963.
Child Process: Released the variable access semaphore.
```

Step 4: After the compilation, please check the p1_result.txt file and get the final answer.



**p1_result.
txt**

Problem 2

1) (5 marks) Have you successfully implemented the program? Has your program passed all the provided 3 test cases? If not, please provide potential reasons for the issues.

Yes, we passed all the provided 3 test cases.

```
Found file: /home/stu/Desktop/project-A-2025/test_case/dir1/file3.txt
Found file: /home/stu/Desktop/project-A-2025/test_case/file1.txt
Found file: /home/stu/Desktop/project-A-2025/test_case/file2.txt
```

```
Child: Word count for /home/stu/Desktop/project-A-2025/test_case/file1.txt: 1
```

```
Child: Word count for /home/stu/Desktop/project-A-2025/test_case/file2.txt: 8
```

```
Child: Word count for /home/stu/Desktop/project-A-2025/test case/dir1/file3.txt: 3
```

2) (5 marks) How you find all the text files under a directory, i.e., the implementation of *traverseDir()* function. (Hint: You may describe the system calls, the data structures, the header files, and any other functions used in your code.)

Assume `dir = /home/stu/Desktop/project-A-2025/test_case`

First, we use `opendir(dir)` to open the directory for searching. Then, by `readdir(dir)` for reading the directory, we use recursive method to do the implementation for finding from `test_case`. If it finds the deeper directory, `snprintf` function will construct the full path into the char array `path`. Then if it tested that is a regular file, which is not a directory, it will check whether it is a `.txt` file. If it does so, we will copy the file into our `File_arr` and tell the user that it finds the file. The base case is to check if the file is current directory or the parent directory, and then it will skip it to avoid the infinite looping. The `stat(path, &states)` part retrieves metadata about the entry at `path`.

```
void traverseDir(char *dir_name){
    // Implement your code here to find out
    // all textfiles in the source directory.

    DIR* dir;
    struct dirent *ent;
    struct stat states;
    char path[512];

    dir = opendir(dir_name);

    while((ent=readdir(dir)) != NULL){
        stat(ent->d_name,&states);
        if(!strcmp(".", ent->d_name) || !strcmp("..", ent->d_name)){
            continue;
        }

        snprintf(path, sizeof(path), "%s/%s", dir_name, ent->d_name);
```

```
220         if (stat(path, &states) == -1) {
221             printf("Cannot stat %s: %s\n", path, strerror(errno));
222             continue;
223         }
224
225         if (S_ISREG(states.st_mode)) {
226             if (num_paths < 10 && strstr(ent->d_name, ".txt")) {
227                 strncpy(File_arr[num_paths], path, 255);
228                 File_arr[num_paths][255] = '\0';
229                 printf("Found file: %s\n", File_arr[num_paths]);
230                 num_paths++;
231             }
232         } else if (S_ISDIR(states.st_mode)) {
233             traverseDir(path);
234         }
235     }
236     closedir(dir);
237 }
```

3) (5 marks) How you achieve the synchronization between two processes using the semaphore.

we create two semaphore for child and parent. When parent is going to do written data, we lock the semaphore for parent, writing the data from .txt file into shared memory. After the writing, parent release the lock for child. Then when child read data from shared memory, it allows the child to read it as there is no lock for shared memory. As a result, it provides the synchronization for avoiding the mutual exclusion.

4) (5 marks) How you handle the case that the total size of a text file exceeds the buffer size.

bytes_read refers to the size of file from reading

```
(SHM_SIZE - 2 * sizeof(sem_t) - 2 * sizeof(int))
```

refers to the size of content.

If the total size of a text file exceeds the buffer size, then it will print the error message.

```
FILE *file = fopen(File_arr[i], "r");
if (!file) {
    printf("Parent: Failed to open file %s: %s\n", File_arr[i], strerror(errno));
    shared->content_size = 0;
}else{
    size_t bytes_read = fread(shared->content, 1, sizeof(shared->content) - 1, file);
    if (ferror(file) || bytes_read > (SHM_SIZE - 2 * sizeof(sem_t) - 2 * sizeof(int))){
        printf("ParentL Error reading file %s or file size exceeds the buffer\n", File_arr[i]);
        shared->content_size = 0;
    }else{
        shared->content[bytes_read] = '\0';
        shared->content_size = (int)bytes_read;
    }
    fclose(file);
}
```

5) (30 marks) **Please declare the chosen student ID number in this problem and attach the screenshot or other proper method to show that your program has find the required txt file and print its name on screen correctly. You should submit all the source files as required in the project instruction documentation and explain the relevant compile and running steps; you should make sure your program runs on virtual machine, and Tutors will test your submitted code on virtual machine and verify the functionality.**

chosen student ID number: 58533046 NG KA HUNG

```

● stu@CS3103:~/Desktop/project-A-2025$ gcc problem2.c helpers.o -o problem2 -I. -pthread
● stu@CS3103:~/Desktop/project-A-2025$ ./problem2 /home/stu/Desktop/project-A-2025/test_case
Found file: /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file6.txt
Found file: /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file5.txt
Found file: /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/file7.txt
Found file: /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file8.txt
Found file: /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file9.txt
Found file: /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file10.txt
Found file: /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file4.txt
Found file: /home/stu/Desktop/project-A-2025/test_case/dir1/file3.txt
Found file: /home/stu/Desktop/project-A-2025/test_case/file1.txt
Found file: /home/stu/Desktop/project-A-2025/test_case/file2.txt
Parent process: My ID is 5145
Parent: Processed /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file6.txt
Parent: Wrote content of /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file6.txt to shared memory
Child process: My ID is 5146

Child: Word count for /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file6.txt: 9
The last number of words in the file is not equal to the last number of chosen student ID 6.
Parent: Processed /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file5.txt
Parent: Wrote content of /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file5.txt to shared memory

Child: Word count for /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file5.txt: 10
The last number of words in the file is not equal to the last number of chosen student ID 6.
Parent: Processed /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/file7.txt
Parent: Wrote content of /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/file7.txt to shared memory

Child: Word count for /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/file7.txt: 142244
The last number of words in the file is not equal to the last number of chosen student ID 6.
Parent: Processed /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file8.txt
Parent: Wrote content of /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file8.txt to shared memory

Child: Word count for /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file8.txt: 22
The last number of words in the file is not equal to the last number of chosen student ID 6.
Parent: Processed /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file9.txt
Parent: Wrote content of /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file9.txt to shared memory

Child: Word count for /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file9.txt: 6
The last number of words in the file is equal to the last number of chosen student ID 6.
Parent: Processed /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file10.txt
Parent: Wrote content of /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file10.txt to shared memory

Child: Word count for /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/dir3/dir4/file10.txt: 17
The last number of words in the file is not equal to the last number of chosen student ID 6.
Parent: Processed /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file4.txt
The last number of words in the file is not equal to the last number of chosen student ID 6.
Parent: Processed /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file4.txt
Parent: Wrote content of /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file4.txt to shared memory

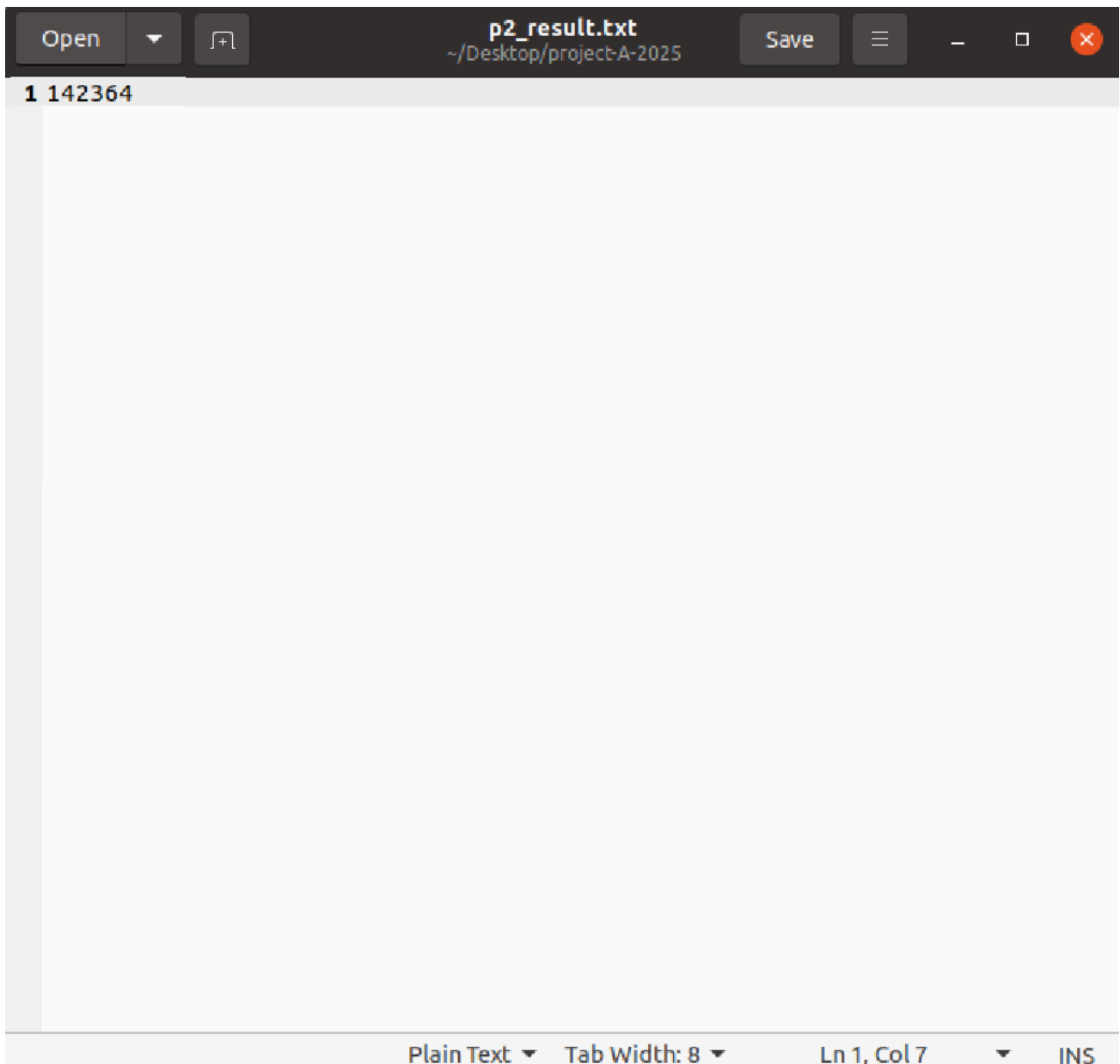
Child: Word count for /home/stu/Desktop/project-A-2025/test_case/dir1/dir2/file4.txt: 44
The last number of words in the file is not equal to the last number of chosen student ID 6.
Parent: Processed /home/stu/Desktop/project-A-2025/test_case/dir1/file3.txt
Parent: Wrote content of /home/stu/Desktop/project-A-2025/test_case/dir1/file3.txt to shared memory

Child: Word count for /home/stu/Desktop/project-A-2025/test_case/dir1/file3.txt: 3
The last number of words in the file is not equal to the last number of chosen student ID 6.
Parent: Processed /home/stu/Desktop/project-A-2025/test_case/file1.txt
Parent: Wrote content of /home/stu/Desktop/project-A-2025/test_case/file1.txt to shared memory

Child: Word count for /home/stu/Desktop/project-A-2025/test_case/file1.txt: 1
The last number of words in the file is not equal to the last number of chosen student ID 6.
Parent: Processed /home/stu/Desktop/project-A-2025/test_case/file2.txt
Parent: Wrote content of /home/stu/Desktop/project-A-2025/test_case/file2.txt to shared memory

Child: Word count for /home/stu/Desktop/project-A-2025/test_case/file2.txt: 8
The last number of words in the file is not equal to the last number of chosen student ID 6.
Child process: Finished.
Parent process: Finished.

```



Explanation:

Step1: Unzip and Get the code

Step2: open VS code for compiling the code

Step3: Compile problem1.c with the provided Makefile:

“\$ gcc -problem2.c -helpers.o -o problem2 -I. -pthread”

```
● stu@CS3103:~/Desktop/project-A-2025$ gcc problem2.c helpers.o -o problem2 -I. -pthread
```

-> If everything goes well, run the executable file with the command:

Examples:

“\$./problem2 /home/stu/Desktop/project-A-2025/test_case”

Step 4: After the compilation, please check the p1_result.txt file and get the final answer.



**p2_result.
txt**